

Mbeya University of Science and Technology

CS 8119/ET 8108: Introduction to High Level Programming

Facilitator: Mr. Magemo A

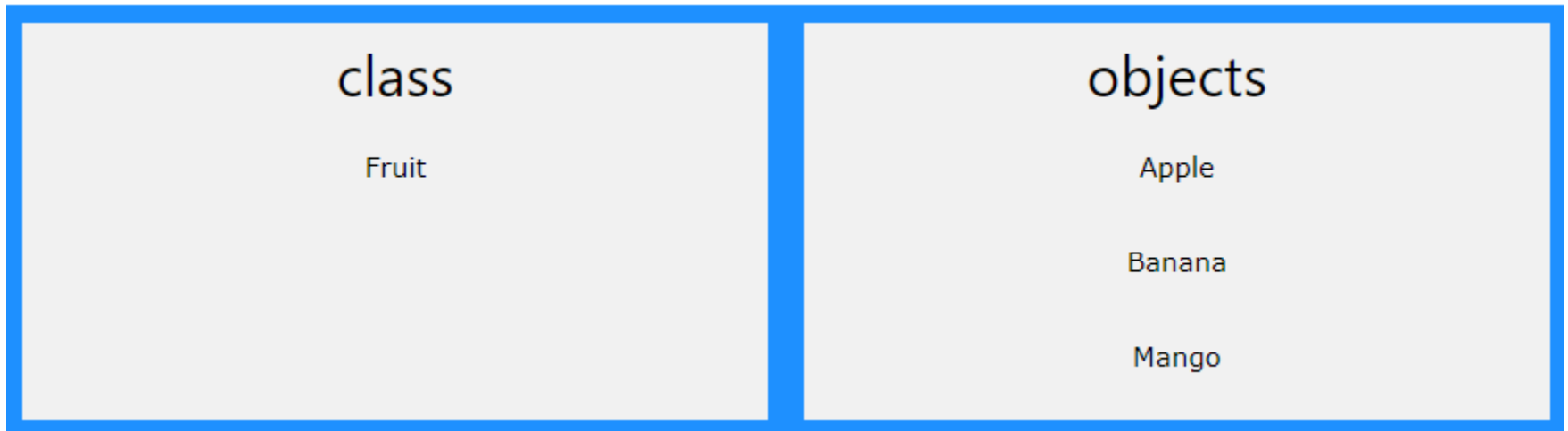
Email: magemohoward30@gmail.com

Lecture 3

◆ Classes and Objects

Classes and objects are the two main aspects of object-oriented programming.

Example;



Classes and Objects

- Class is a template for objects, and an object is an instance of a class.
- When the individual objects are created, they inherit all the variables and functions from the class.

Classes and Objects

- Everything in C++ is associated with classes and objects, along with its attributes and methods.
- For example: in real life, a car is an object.
- The car has **attributes**, such as **weight** and **color**, and **methods**, such as **drive** and **brake**.
- Attributes and methods are basically variables and functions



Classes and Objects

Create a Class

To create a class, use the class keyword:

Example

Create a class called "MyClass":

```
class MyClass {           // The class
    public:                // Access specifier
        int myNum;         // Attribute (int variable)
        string myString;   // Attribute (string variable)
};
```

Classes and Objects

Create a Class

- ❖ The **class** keyword is used to create a class called **MyClass**.
- ❖ The **public** keyword is an access specifier, which specifies that members (attributes and methods) of the class are accessible from outside the class. You will learn more about access specifiers later.
- ❖ Inside the class, there is an integer variable **myNum** and a string variable **myString**. When variables are declared within a class, they are called attributes.
- ❖ At last, end the class definition with a semicolon ;.

Classes and Objects

To Create an Object

- Object is created from a class
- To create an object of `MyClass`, specify the **class name**, followed by the **object name**.
- To access the class attributes (`myNum` and `myString`), use the dot syntax (`.`) on the object:

Classes and Objects

To Create an Object

```
class MyClass {  
    public:  
        int age;  
        string name;  
};
```

```
int main() {  
    MyClass b; // Create an object of MyClass  
  
    // Access attributes and set values  
    b.age = 15;  
    b.name = "Some text";  
  
    // Print attribute values  
    cout << b.age << "\n";  
    cout << b.name;  
    return 0;  
}
```

You can create many object as you can.....

Classes and Objects

Class Methods

➤ Methods are functions that belongs to the class.

There are two ways to define functions that belongs to a class:

1. Inside class definition
2. Outside class definition

In the following example, we define a function inside the class, and we name it "myMethod".

◆ Classes and Objects

To Create a method inside class

```
class MyClass {  
    public:  
        void myMethod() {  
  
// Method/function defined inside the class  
  
        cout << "Hello World!";  
    }  
};
```

```
int main() {  
    // Create an object of MyClass  
    MyClass myObj;  
    // Call the method  
    myObj.myMethod();  
    return 0;  
}
```

◆ Classes and Objects

To Create a method outside class

```
class MyClass {  
    public:  
        void myMethod(); // Method/function  
        declaration  
};  
  
// Method/function defined outside the class  
void MyClass::myMethod()  
{  
    cout << "Hello World!";  
}
```

```
int main() {  
    // Create an object of MyClass  
    MyClass myObj;  
    // Call the method  
    myObj.myMethod();  
    return 0;  
}
```

◆ Classes and Objects

Example with parameters

```
#include <iostream>
using namespace std;

class Car {
public:
    int speed(int maxSpeed);
};

int Car::speed(int maxSpeed) {
    return maxSpeed;
}
```

```
int main() {
    Car myObj; // Create an object of Car
    cout << myObj.speed(200); // Call the
    method with an argument
    return 0;
}
```

◆ Classes and Objects

Constructors

- is a special method that is automatically called when an object of a class is created.
- To create a constructor, *use the same name as the class*, followed by *parentheses ()*:



Classes and Objects

Example

```
class MyClass {  
    public:  
        MyClass() {    // Constructor  
            cout << "Hello World!";  
        }  
};  
  
int main() {  
    MyClass myObj; // Create an object of MyClass (this will call the  
    constructor)  
    return 0;}
```



Classes and Objects

Constructor with Parameters

```
class Car {           // The class
    public:           // Access specifier
        string brand; // Attribute
        string model; // Attribute
        int year;     // Attribute

        Car(string x, string y, int z) { // Constructor with parameters
            brand = x;
            model = y;
            year = z;
        }
};
```



Classes and Objects

Constructor with Parameters

```
int main() {  
    // Create Car objects and call the constructor with different values  
    Car carObj1("BMW", "X5", 1999);  
    Car carObj2("Ford", "Mustang", 1969);  
  
    // Print values  
    cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";  
    cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";  
    return 0;  
}
```


◆ Classes and Objects

Access Specifiers

- Access specifiers define how the members (attributes and methods) of a class can be accessed.
- In the example below, the members are public - which means that they can be accessed and modified from outside the code.

```
class MyClass {  
  
    public:        // Access specifier  
                  // class members goes here  
};
```

◆ Classes and Objects

There are three access specifiers:

1. **public** - members are accessible from outside the class
2. **private** - members cannot be accessed (or viewed) from outside the class
3. **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes.

Classes and Objects

Example: difference between private & public member

```
class MyClass {  
    public:    // Public access specifier  
        int x; // Public attribute  
    private:  // Private access specifier  
        int y; // Private attribute  
};  
  
int main() {  
    MyClass myObj;  
    myObj.x = 25; // Allowed (public)  
    myObj.y = 50; // Not allowed (private)  
    return 0;  
}
```

If you try to access a private member, an error occurs:
error: y is private

Encapsulation

- The meaning of Encapsulation, is to make sure that "sensitive" data is hidden from users.
- To achieve this, you must declare class variables/attributes as private (cannot be accessed from outside the class).
- If you want others to read or modify the value of a private member, you can provide public **get** and **set** methods.



Encapsulation Example

```
#include <iostream>
using namespace std;

class Employee {
private:
    // Private attribute
    int salary;

public:
    // Setter
    void setSalary(int s) {
        salary = s;
    }
    // Getter
    int getSalary() {
        return salary;
    }
};
```

```
int main() {
    Employee myObj;
    myObj.setSalary(50000);
    cout << myObj.getSalary();
    return 0;
}
```

Why Encapsulation?

1. Encapsulation ensures better control of your data, because you (or others) can change one part of the code without affecting other parts
2. Increased security of data

Inheritance

- Inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically.
- The class which inherits the members of another class is called **derived class** and the class whose members are inherited is called **base class**.
- **Advantage is Code reusability**: you can reuse the members of your parent class. there is no need to define the member again. So less code is required in the class.



Inheritance

Types Of Inheritance

1. Single inheritance
2. Multiple inheritance
3. Hierarchical inheritance
4. Multilevel inheritance
5. Hybrid inheritance



Inheritance

Derived Classes

A Derived class is defined as the class derived from the base class.

The Syntax of Derived class:

```
class derived_class_name :: visibility-mode base_class_name
{
    // body of the derived class.
}
```



Inheritance

Derived Classes

derived_class_name: It is the name of the derived class.

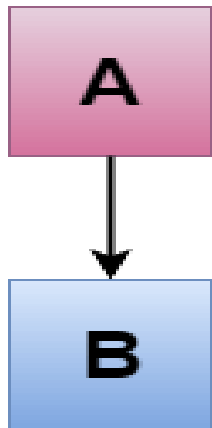
visibility mode: The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited

base_class_name: It is the name of the base class.

◆ Inheritance

Single Inheritance

- Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

Inheritance

Single Level Inheritance Example

When one class inherits another class, it is known as single level inheritance.

See the next example

Inheritance

Example

```
#include <iostream>
using namespace std;
class Account {
public:
    float salary = 60000;
};
class Programmer: public Account {
public:
    float bonus = 5000;
};
int main(void) {
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Account is the base class and **Programmer** is the derived class.

Inheritance

Example

```
#include <iostream>
using namespace std;
class Animal {
    public:
    void eat() {
        cout<<"Eating..."<<endl;
    }
};

class Dog: public Animal
{
    public:
    void bark(){
        cout<<"Barking...";
    }
};
```

```
int main(void) {
    Dog d1;
    d1.eat();
    d1.bark();
    return 0;
}
```

**Inheriting
Methods.**

Polymorphism

- Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.
- Inheritance lets us inherit attributes and methods from another class. Polymorphism uses those methods to perform different tasks. This allows us to perform a single action in different ways.

Polymorphism

- For example, think of a base class called `Animal` that has a method called `animalSound()`. Derived classes of `Animals` could be `Pigs`, `Cats`, `Dogs`, `Birds` - And they also have their own implementation of an animal sound (the pig oinks, and the cat meows, etc.):

Polymorphism

Example

```
// Base class
class Animal {
public:
    void animalSound() {
        cout << "The animal makes a sound
\n" ;
    }
};

// Derived class
class Pig : public Animal {
public:
    void animalSound() {
        cout << "The pig says: wee wee \n" ;
    }
};
```

Polymorphism

Example

```
// Derived class
class Dog : public Animal {
public:
    void animalSound() {
        cout << "The dog says: bow wow \n" ;
    }
};

int main() {
    Animal myAnimal;
    Pig myPig;
    Dog myDog;

    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
    return 0;
}
```

Pig and **Dog** objects are created and override the **animalSound()** method:

ASSIGNMENT

1. What is Abstraction/
2. Write the c++ program exhibiting abstraction concept
3. Develop a program that will be able to create, write and read to a file
4. Develop a program that will throw an exception when an error occur and should have a block of code to handle the error.
5. Use the concept of function overloading to write the program that will perform an average of three numbers, display the name of three subject and calculate area of a circle.
6. Write simple program for , Linked lists, stacks and queues
7. What is a Aggregation? Use Staff and student classes to implement Aggregation concept.

Thank you for Listening! Questions

